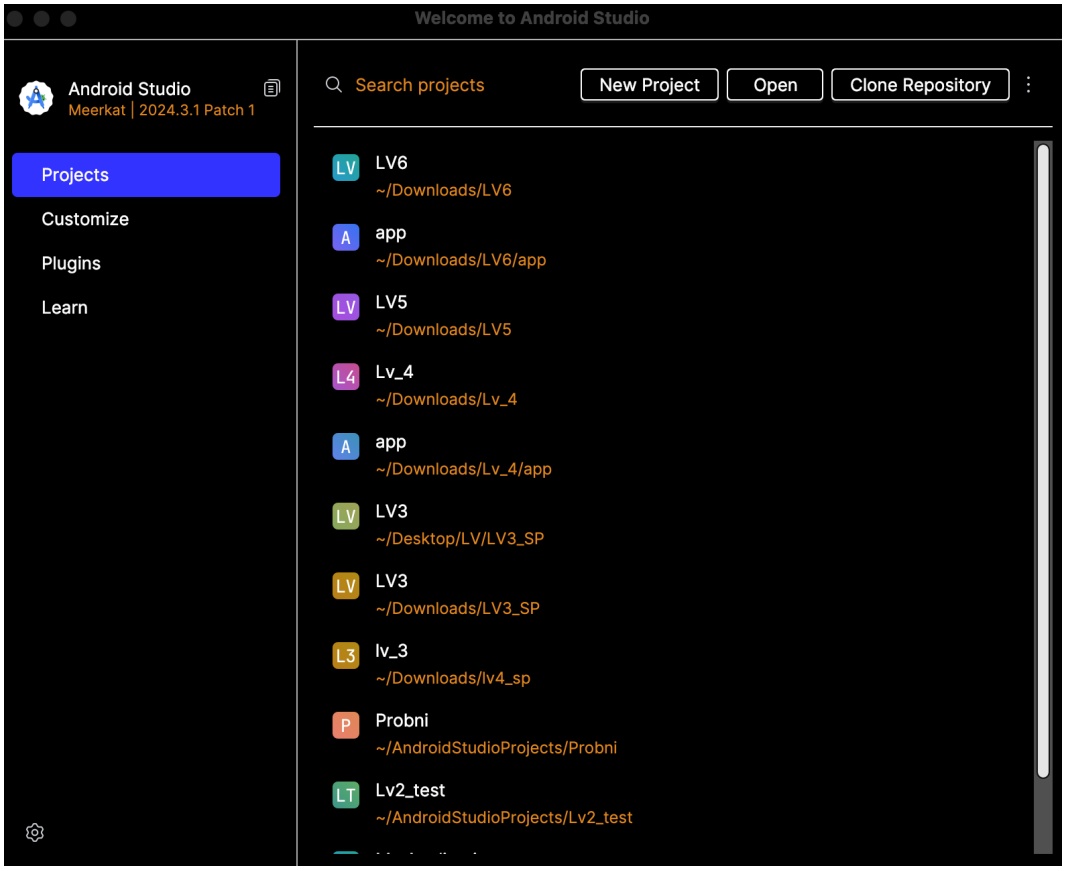
RAZVOJ MOBILNIH APLIKACIJA:

LV 1: Napredni Jetpack Compose

1. Stvaranje prve Android aplikacije

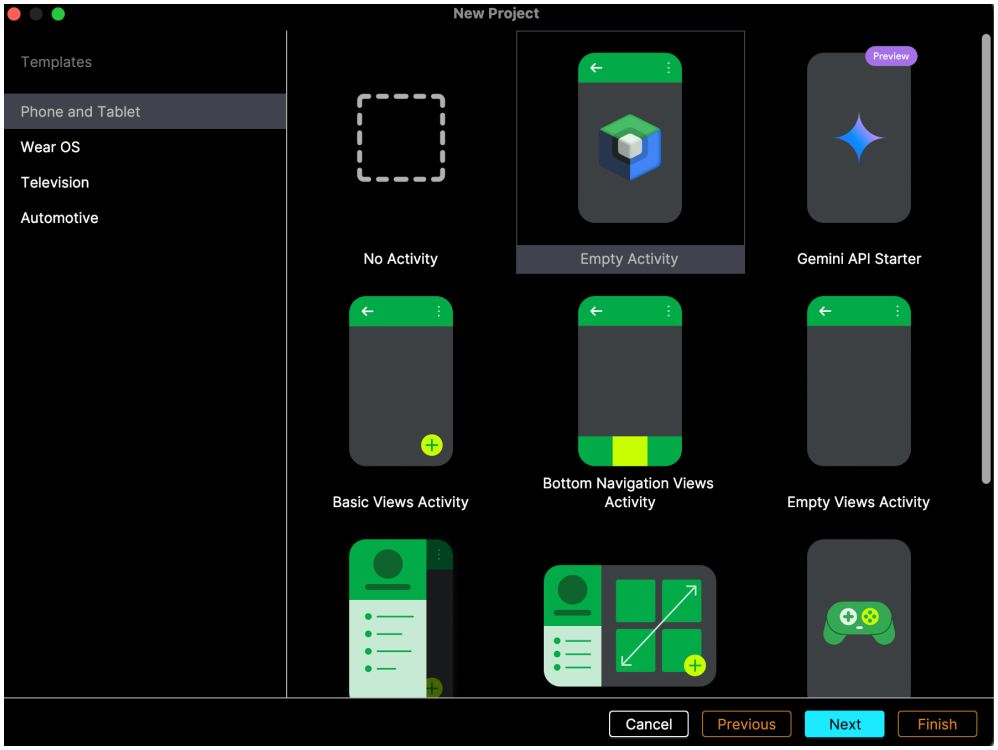
Tijekom laboratorijskih vježbi koristit ćemo Android Studio kao glavno razvojno okruženje za izradu Android aplikacija. Sve vježbe su pripremljene u verziji **Android Studio Meerkat**, stoga je preporučljivo instalirati ovu verziju kako biste izbjegli moguće razlike u sučelju ili funkcionalnostima. Nakon pokretanja Android Studija, otvoriti će se početni zaslon prikazan na slici 1. Ovaj zaslon služi kao polazna točka za stvaranje novih projekata ili otvaranje postojećih. Kako bismo započeli rad na našoj prvoj aplikaciji, kliknite na gumb "New Project".



Slika 1. *Početni zaslon Android studio Meerkat*

Klikom na "New Project" otvara se prozor za odabir vrste projekta, kao što je prikazano na slici 2. Ovdje ćemo odabrati **"Empty Activity"**. Ova opcija predstavlja najjednostavniji predložak za Android aplikaciju – praznu aktivnost koja služi kao osnovni zaslon bez unaprijed definiranog sadržaja. "Empty Activity" idealna je početna točka za učenje jer nam omogućava da sami

kreiramo korisničko sučelje od nule, koristeći alate poput Jetpack Composea. Nakon što odaberete "Empty Activity", kliknite na gumb **"Next".**

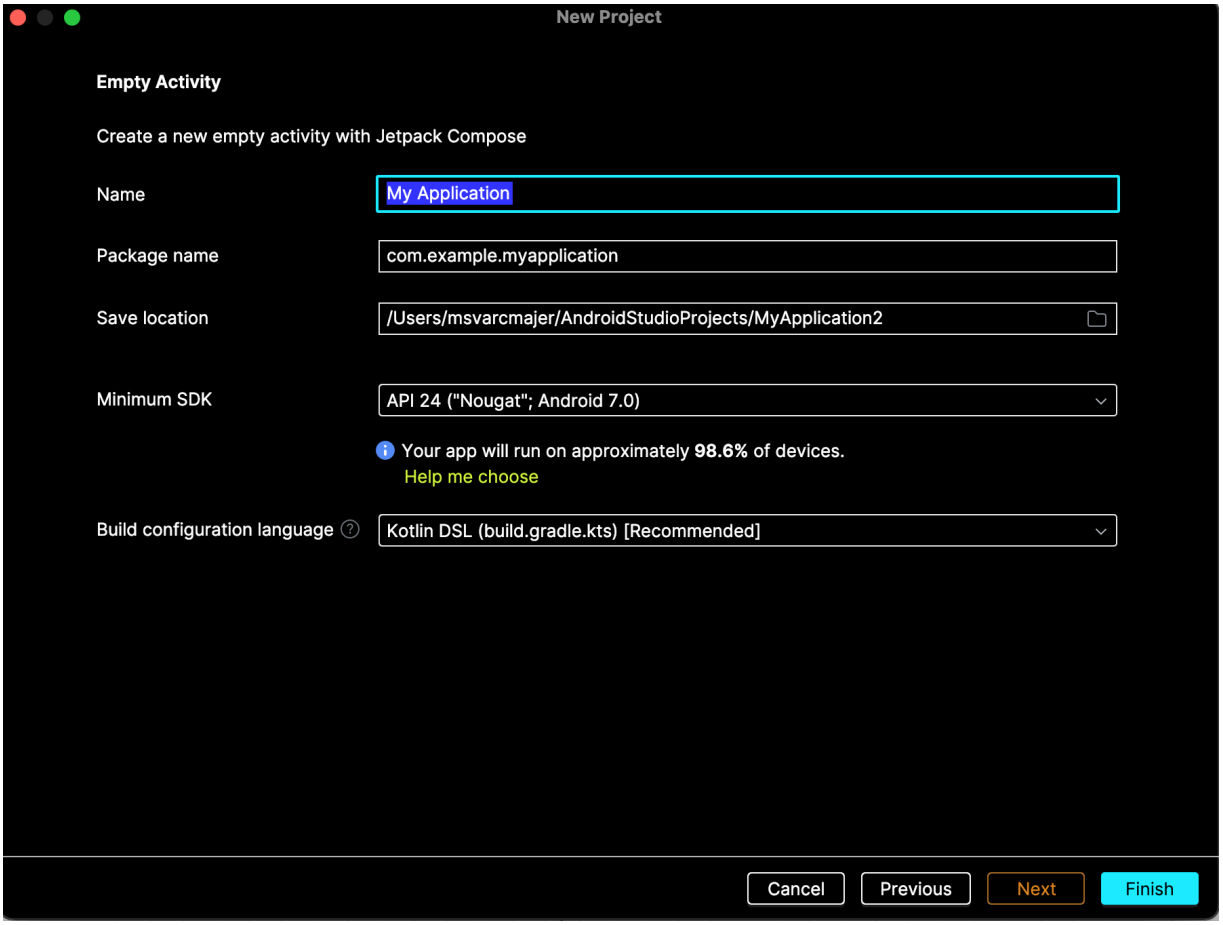


Slika 2. *Odabir vrste projekta*

Nakon klika na "Next", otvorit će se prozor za konfiguraciju projekta, prikazan na slici 3. Ovdje definiramo osnovne postavke aplikacije:

- **Name**: Unesite naziv projekta, primjerice "prezime_LV2". Ovaj naziv ne mora biti jedinstven i služi samo za vašu osobnu identifikaciju projekta u Android Studiju.
    - **Package Name**: Ovo je jedinstveni identifikator aplikacije. Preporučuje se korištenje obrnutog oblika domene (npr. com.example.prezime_LV1).
    - **Save Location**: Odaberite direktorij na računalu gdje želite spremiti projekt.
    - **Minimum SDK**: Postavite na **API 24 (Android 7.0, Nougat)**. Ovo osigurava da aplikacija radi na većini modernih uređaja, uz dobar balans između kompatibilnosti i dostupnih funkcionalnosti.

Kada završite s postavljanjem, kliknite na **"Finish"**. Android Studio će tada stvoriti projekt i sinkronizirati potrebne datoteke (Gradle build sustav), što može potrajati nekoliko minuta.



Slika 3. *Konfiguracija projekta*

2. Jetpack compose

Jetpack Compose moderni je UI toolkit za razvoj aplikacija na Android platformi. Radi se o deklarativnom pristupu izgradnji korisničkog sučelja (UI), gdje se koristi Kotlin kod za opisivanje kako bi UI trebao izgledati i ponašati se u različitim situacijama. Umjesto pisanja zasebnih datoteka za strukturu i logiku, Compose omogućava definiranje cijelog korisničkog sučelja unutar jednog dijela koda, čime se postiže intuitivan i reaktivni programski model.

Osnovni građevni blokovi u Jetpack Composeu su **Composable funkcije**. Svaka Composable funkcija opisuje dio korisničkog sučelja (npr. tekst, gumb, sliku) i može se sastojati od drugih

Composable funkcija, omogućujući stvaranje složenih hijerarhija korisničkog sučelja na jednostavan način.

2.1. Composable funkcije

Composable funkcije su osnovni građevni blokovi korisničkog sučelja u Jetpack Composeu. Composable Funkcije:

- Opisuju dio korisničkog sučelja koji se prikazije na zaslonu (npr. Tekst, gumb, sliku)
- Ne vraćaju nikakvu vrijednost
- Primaju parametre i generiraju prikaz na temelju tog unosa

Programski kod 1. prikazuje deklaraciju composable funkcije. Svaka Composable funkcija mora biti označena anotacijom @Composable, koja označava da funkcija može stvarati UI elemente. Također, često se koristi anotacija @Preview za prikazivanje UI-a u Android Studiju bez pokretanja aplikacije.

```
@Preview(showBackground = true)
@Composable
fun Greeting(name: String) {
    Text(text = "Pozdrav, $name!")
}
```

Programski kod 1. *Primjer composable funkcije*

Composable funkcija mora slijediti konvenciju imenovanja **PascalCase**. Ova konvencija zahtijeva da prvo slovo svake riječi u složenom imenu bude veliko (npr. Greeting, UserProfile).

Razlike u stilovima imenovanja

- **PascalCase**: Sva prva slova riječi su velika (npr. DoneButton).
- **CamelCase**: Prva riječ počinje malim slovom, a ostale riječi velikim (npr.

doneButton – ne koristi se za Composable funkcije).

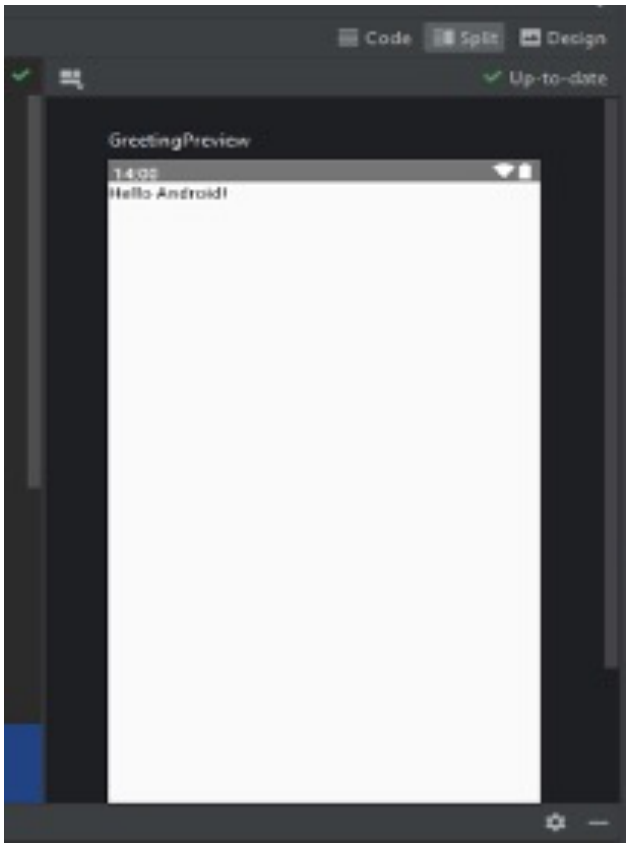
Pravila za imenovanje

Funkcija mora biti imenica: DoneButton()

- **Ne smije biti glagol ili glagolska fraza**: DrawTextField()
- **Ne smije biti prijedlog s imenicom**: TextFieldWithLink()
- **Ne smije biti pridjev**: tamno()
- **Ne smije biti prilog**: izvana()
- **Dozvoljeno je dodavanje opisnih pridjeva kao prefiks**: RoundIcon()

2.2. Okno dizajna u Android studiju

Android Studio omogućuje pregled Composable funkcija izravno unutar IDE-a, bez potrebe za pokretanjem aplikacije na emulatoru ili fizičkom Android uređaju. Ova značajka štedi vrijeme tijekom razvoja jer omogućuje brzo testiranje i prilagođavanje korisničkog sučelja (UI). Pregled je dostupan u oknu dizajna (Design pane), koje se nalazi u uređivaču koda, a prikazano je na slici 3.



Slika 3. Okno dizajna

2.3. Hijerarhija korisničkog sučelja u Jetpack Composeu

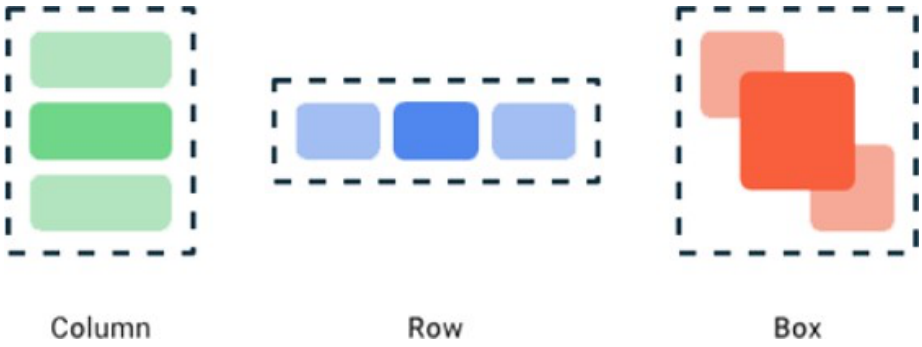
U Jetpack Composeu korisničko sučelje (UI) organizirano je hijerarhijski putem layout komponenata koje sadrže druge UI elemente. Pojmovi **roditelj** (parent) i **dijete** (child) opisuju ovu strukturu:

- Roditeljski element (npr. Column, Row ili Box) sadrži jedan ili više podređenih elemenata (npr. Text ili Button).
- Podređeni elementi mogu sami biti roditelji drugih elemenata, stvarajući složenu hijerarhiju.

Osnovne layout komponente za raspored elemenata uključuju:

- **Column**: Raspoređuje elemente vertikalno
- **Row**: Raspoređuje elemente horizontalno
- **Box**: Omogućuje preklapanje elemenata

Na slici 4 prikazani su primjeri rasporeda elemenata koristeći Column, Row i Box, dok programski kod 2. prikazuje primjer korištenja Column komponente.



Slika 4. Elementi smješteni u reda, stupac i box

```
Column {
    Text(
        text = "Hello Miljenko"
    )
    val bmi = 15
    Text(
        text = "Tvoj BMI je: $bmi"
    )
}
```

Programski kod 2. Primjer Column komponente

Osnovne komponente intuitivne, fleksibilne i odlične za većinu jednostavnijih rasporeda. No kako aplikacije postaju kompleksnije, raste i potreba za učinkovitijim prikazom većeg broja elemenata, boljom kontrolom pozicioniranja ili automatskim prilagođavanjem sadržaja dostupnom prostoru.

Programskim kodom 3, dan je primjer liste koju ćemo ubaciti u naš kod ispod zadnjeg teksta.

```
val vjezbe = listOf(
        "Trčanje", "Plivanje", "Biciklizam", "Planinarenje",
        "Skakanje užeta", "Veslanje", "Joga", "Pilates",
        "Vježbe snage", "Pliometrija", "Brzo hodanje"
    )
```

Programski kod 3. *Primjer liste*

Napredne layout komponente su:

- **LazyColumn**: Lista elemenata prikazana vertikalno
- **LazyRow**: Horizontalna lista elemenata

- **ConstraintLayout:** Napredna kontrola položaja – slična XML ConstraintLayoutu
- **LazyVerticalGrid:** Prikaz u mreži (grid) – korisno za slike, kartice, itd.
- **FlowRow / FlowColumn:** Automatski prelamanje sadržaja kad nestane prostora

Programski kod 3 sadrži listu od 11 elemenata. Korištenje Columna ovdje nije problem, međutim da se radi o listi sa stotinama ili tisućama elemenata (a prikazati ih se može samo 10 na ekranu), bitno je racionalno koristiti memoriju pa učitavati elemente samo kada ih trebamo. Za ovakve slučajeve možemo koristiti naprednije layout komponente. U ovom slučaju LazyColumn kako je prikazano kodom 4.

```
LazyColumn(
        modifier = modifier,
        contentPadding = PaddingValues(16.dp) // da ima lijepi
razmak od ruba
    ) {
        item {
            Text(
                text = "Hello Miljenko",
                fontSize = 20.sp
            )
        }
        item {
            Text(
                text = "Tvoj BMI je: $bmi",
                fontSize = 20.sp
            )
        }
        item {
            Text(
                text = "Preporučene vježbe su:",
                fontSize = 20.sp
            )
        }
        items(vjezbe) { vjezba ->
            Text(
                text = vjezba,
                fontSize = 18.sp
            )
        }
    }
}
```

Programski kod 4. Primjer LazyColumn

Obavezno: Više o layout komponentama

Responzivno skaliranje UI elemenata: DP i SP u praksi

Dodali ste tekst u svoje korisničko sučelje, ali još ne izgleda kao konačna aplikacija. Potrebno je promijeniti veličinu, boju teksta i druge atribute koji utječu na izgled elementa teksta. Također možete eksperimentirati s različitim veličinama i bojama fonta.

U Android aplikacijama koriste se dvije glavne mjerne jedinice za veličinu elemenata:

DP (Density-Independent Pixels): Jedinica neovisna o gustoći zaslona, korištena za dimenzije elemenata poput razmaka ili širine.

SP (Scalable Pixels): Skalabilni pikseli specifični za veličinu fonta. Po zadanim postavkama, SP je isti kao DP, ali se prilagođava korisnikovim postavkama veličine teksta na telefonu (npr. povećanje teksta za osobe s lošim vidom). Jetpack compose ne dozvoljava korištenje DP za veličinu fonta.

EM (relativna mjera): Relativna veličina prema trenutnom default fontu.

PX (pixeli): Ne preporuča se u Jetpack Composeu.

Za prilagođavanje veličine teksta u Jetpack Composeu preporučuje se korištenje SP jer omogućava fleksibilnost i pristupačnost. Da biste koristili SP, potrebno je uvesti paket androidx.compose.ui.unit.sp.

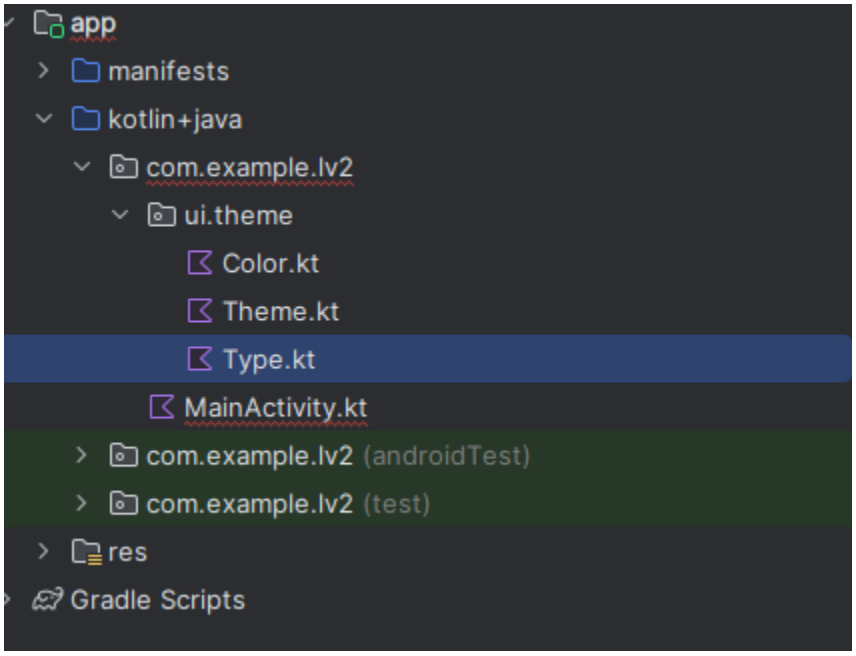
Veličina fonta definirana je parametrom fontSize unutar Text komponente. Programskim kodom 5, određene su različite veličine fonta. Testiranjem na tabletu, može se vidjeti razlika.

```
Text(
            text = "Hello Miljenko",
            fontSize = 20.sp,
            modifier = modifier
        )
        val bmi = 99
        Text(
            text = "Tvoj BMI je: $bmi",
            fontSize = 20.dp,
            modifier = modifier
        )
```

Programski kod 5. Razlike u određivanju veličine fonta

2.4. Korištenje novog fonta

Do sada smo radili uglavnom s datotekom MainActivity.kt, ali Android projekat sadrži mnoge druge datoteke koje podržavaju funkcionalnost i dizajn aplikacije. Organizaciju datoteka možete vidjeti na slici 5. Posebno je važan folder ui.theme, koji sadrži datoteke poput Color.kt, theme.kt, Type.kt. fontove mijenjamo u datoteci Type.kt.



Slika 5. *Organizacija datoteka*

Ako želite koristiti vlastiti font (npr. preuzet s interneta), možete ga dodati u projekt na sljedeći način:

Preuzmite datoteku fonta u formatu .ttf ili .otf (npr. sa stranica poput Google Fonts).

1. U folderu res (resources) vašeg projekta stvorite novi podfolder pod imenom **font** (npr. app/src/main/res/font).
2. Kopirajte preuzetu datoteku fonta (npr. custom_font.ttf) u taj folder.
3. U datoteci Type.kt, definirajte vlastiti font koristeći FontFamily i referencirajte datoteku fonta.

Programski kod 4 prikazuje kako dodati vlastiti font u Type.kt

```
val CustomFont = FontFamily(
    Font(R.font.custom_font)
)
// Set of Material typography styles to start with
val Typography = Typography(
    bodyLarge = TextStyle(
        fontFamily = CustomFont,
        fontWeight = FontWeight.Normal,
        fontSize = 16.sp,
        lineHeight = 24.sp,
        letterSpacing = 0.5.sp
    )
)
```

Programski kod 6. *Dodavanje custom fonta*

2.5. Lokalizacija aplikacije

Lokalizacija (Localization, L10n) omogućuje da se aplikacija prilagodi jeziku, regiji i kulturi korisnika. Umjesto da se tekstovi u aplikaciji pišu direktno u kodu, oni se definiraju u vanjskim resursima (strings.xml), što omogućuje jednostavno dodavanje prijevoda za više jezika.

Glavne komponente lokalizacije:

- Tekstovi se ne pišu direktno u kodu već u strings.xml datotekama.
- Dodaju se posebne verzije resursa za svaki jezik (npr. values-hr za hrvatski).
- Android sustav automatski odabire odgovarajući prijevod ovisno o jeziku postavljenom na uređaju.
- Formati za datum, vrijeme i brojeve također se mogu prilagoditi različitim

regijama.

Struktura resursa:

- res/values/strings.xml — osnovni (default) jezik aplikacije (najčešće engleski).
- res/values-hr/strings.xml — hrvatski jezik.
- res/values-de/strings.xml — njemački jezik itd.

Slika 6. Dodavanje novog jezika u aplikaciju

U filename je potrebno upisati strings.xml. Nakon toga, potrebno je pod Availible qualifiers odabrati Locale a potom koristeći >> prebaciti odabrano pod Chosen quailifiers. Pod Language odabrati Hrvatski. Android studio će automatski popuniti Directory name.

Isti postupak proći i sa njemačkim jezikom.

Programski kod 7 prikazuje izgled datoteke strings.hml.

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="hello_miljenko">Hello Miljenko</string>
    <string name="your_bmi">Your BMI is: %1$d</string>
    <string name="recommended_exercises">Recommended
exercises:</string>
</resources>
```

Programski kod 7. *Default strings.xml*

Programski kod 8 prikazuje izgled datoteke strings.xml(de-rDE).

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="hello_miljenko">Hallo Miljenko</string>
    <string name="your_bmi">Dein BMI ist: %1/string>
    <string name="recommended_exercises"> Empfohlene
Übungen:</string>
</resources>
```

Programski kod 8. *Njemački strings.xml*

Programski kod 9 prikazuje izgled datoteke strings.xml(hr-rHR).

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="hello_miljenko">Pozdrav Miljenko</string>
    <string name="your_bmi">Tvoj BMI je: %1$d</string>
    <string name="recommended_exercises">Preporučene
vježbe:</string>
</resources>
```

Programski kod 9. *Hrvatski strings.xml*

Korištenje slika

Slike su važan dio korisničkog sučelja aplikacija jer poboljšavaju vizualni dojam i korisničko iskustvo. Prilikom odabira i postavljanja slika u Android aplikaciji potrebno je obratiti pažnju na **namjenu slike**, **format** i **gustoću** ekrana.

Odabir slike za različite namjene

- **Slika unutar aplikacije**
  - Najčešće su to male, kvadratne slike.
  - Preporučeni format: **PNG** (zbog prozirne pozadine) ili **JPG** (ako nema prozirnosti).
  - Dimenzije: oko **128x128 px** za standardne ekrane.
  - Treba osigurati slike različitih gustoća (dpi) za različite uređaje:
    - **mdpi**: 128x128 px
    - **hdpi**: 192x192 px
    - **xhdpi**: 256x256 px
    - **xxhdpi**: 384x384 px
    - **xxxhdpi**: 512x512 px
- **Pozadinska slika**
  - Obično su to slike velikih dimenzija koje zauzimaju cijeli ekran ili veći dio sučelja.
  - Preporučeni format: **JPG** (manja veličina datoteke), može i **WEBP** (noviji, optimiziran format).
  - Dimenzije ovise o ciljanom uređaju, npr. za mobitele oko **1080x1920 px**.
  - Treba koristiti **različite verzije** za različite gustoće ako je pozadinska slika lokalna (ako koristiš mrežnu pozadinu – nije potrebno).

Unutar aplikacije sliku dodajemo pomoću programskog koda 10. pokretanja aplikacije.

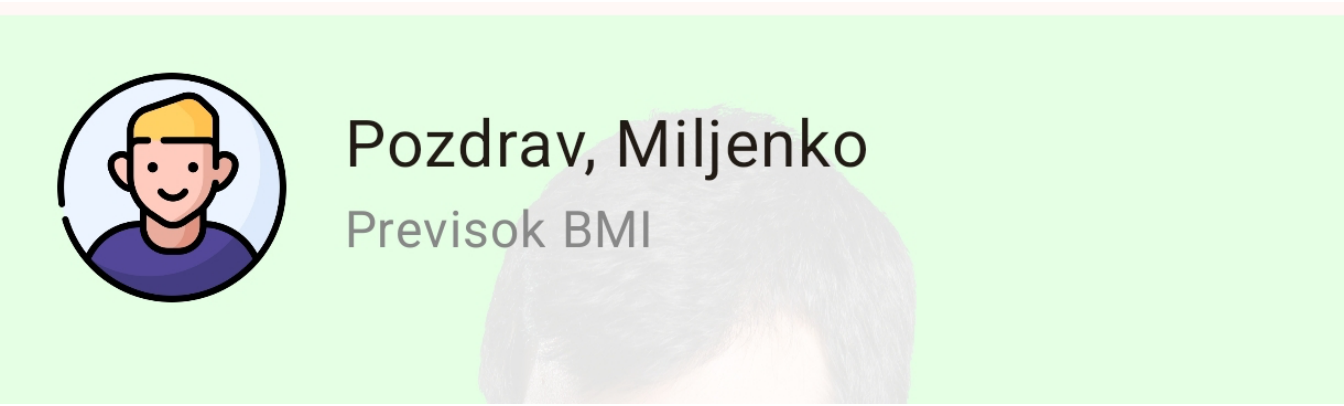
```
Image(

    painter = painterResource(id
    = R.drawable.profile_pic),

    contentDescription ="Profilna
    slika modifier=
    Modifier.size(128.dp)"

)
```

Programski kod 10. *Dodavanje slike*

3. Zadatak za samostalni rad- Izrada korisničkog profila

Opis zadatka:

Potrebno je izraditi novu Composable funkciju naziva `UserPreview` koja prikazuje korisnički profil uz izračun i prikaz indeksa tjelesne mase (BMI).

Zahtjevi:

- Funkcija mora nositi naziv: **`UserPreview`**.
- Funkcija mora primati sljedeće parametre:
  - `heightCm: Int` – visina korisnika u centimetrima,
  - `weightKg: Int` – težina korisnika u kilogramima.
- U funkciji je potrebno izračunati **BMI**.
- Na temelju izračunatog BMI-a, treba prikazati jednu od tri moguće poruke:
  - Ako je BMI < 18.5 ➔ "Prenizak BMI"
  - Ako je BMI između 18.5 i 24.9 ➔ "Idealan BMI"
  - Ako je BMI ≥ 25 ➔ "Previsok BMI"
- Izgled prikaza treba biti sljedeći:
  - **Pozadina:**
    - Koristiti sliku `androidparty` iz predloška kao pozadinsku sliku.
    - Postaviti prozirnost (alfa) na **0.1** tako da slika bude jedva vidljiva.
  - **Glavni sadržaj:**
    - Postaviti **profilnu sliku** s lijeve strane (slika `profile_pic`).
      - Veličina slike: **64.dp**.
      - Oblik slike: **krug**.
    - S desne strane slike postaviti:
      - Gornji red: Tekst "Pozdrav, Miljenko" (ime može biti hardkodirano).
      - Donji red: Poruka o BMI statusu (ovisno o izračunu).
- Profilna slika i tekstovi trebaju biti **centrično poravnati** vertikalno.
- Aplikacija mora biti lokalizirana za hrvatski i engleski